

## מעבדה מספר 6

### OCSF

מטרת המעבדה היא לעשות שימוש בתוכנת OCSF ליצירת אפליקציה פשוטה של Client-Server.

הקובץ המובא לפניכם כולל הסבר על מבנה התוכנה בה נעשה שימוש ובסופו משימות לביצוע. בנוסף מצורף קובץ ZIP של קוד תוכנת ה-OCSF ושל אפליקצית SimpleChat המאפשרת החלפת הודעות בין Client ל-Server תוך שימוש ב-OCSF.

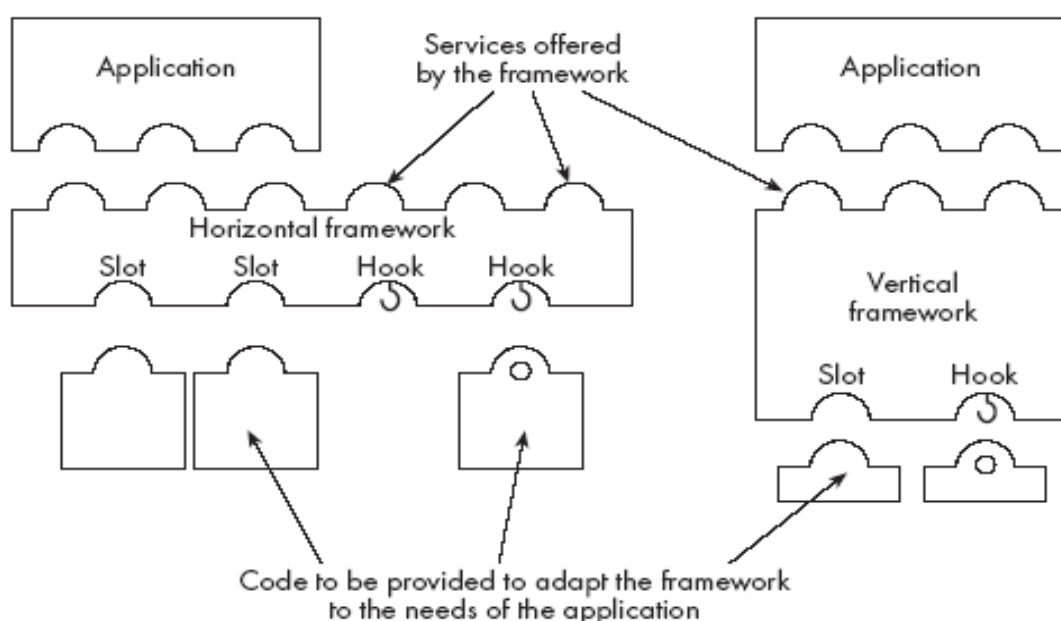
### 1. Frameworks

Framework היא תוכנה ברת שימוש חוזר אשר מממשת פתרון ג'נרי לבעיה כללית. היא מספקת שירותים שניתן ליישם בתוכניות שונות. השימוש בה מבוסס על העקרון שאפליקציות תוכנה הן שונות אבל יש בהן מן המשותף. במהותה ה-Framework אינה שלמה.

- היא משתמשת במחלקות ובמתודות שחסרות בה (slots).
- חלק מהפונקציונאליות היא אפשרית – המשתמש יכול להוסיפה (חלקים אלה נקראים hooks או extension points).
- מפתחים משתמשים בשירותים שה-framework מספקת (שירותים אלה נקראים API – Application Program Interface).
- בגישה של תכנות מונחה עצמים ה-framework מורכבת מספריה של מחלקות. ה-API מגדיר קבוצה של מתודות ציבוריות של המחלקות. חלק ממחלקות אלה תהיינה אבסטרקטיות ובנוסף הוא כולל ממשקים.

### סוגים של frameworks

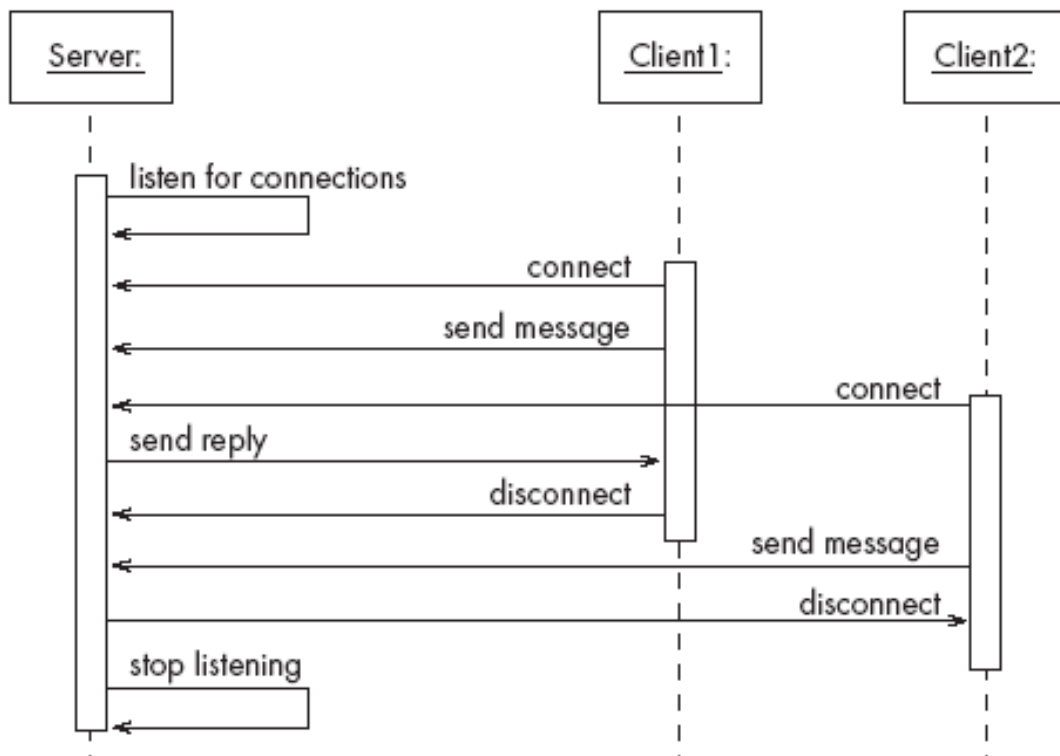
- horizontal framework – מספקת שירותים כלליים שיכולים להיות שימושיים למספר גדול של אפליקציות.
- Vertical framework – היא יותר "שלמה" אבל עדיין כוללת מספר slots שיש להשלים כדי להתאימה לצרכים של אפליקציה מסויימת.



## 2. ארכיטקטורה של Client-Server

מערכת מבוצרת היא מערכת אשר:  
 החישובים בה מתבצעים בתוכניות נפרדות.  
 על פי רוב מתבצעת על יחידות חומרה שונות שמשתפות פעולה כדי לבצע משימה של המערכת.  
 ה-Server היא תוכנית שירותים לתוכניות אחרות המחוברות אליה ע"י שימוש בערוץ תקשורת.  
 ה-Client היא תוכנית שנגשת ל-server (לאחד או יותר) כדי לקבל שירותים.  
 ה-server יכול לספק שירותים לכמה clients במקביל.  
**דוגמאות:** רשת האינטרנט, Email, מערכות תקשורת, מערכת של database משותף למספר מחשבים.

### תקשורת של server עם שני clients

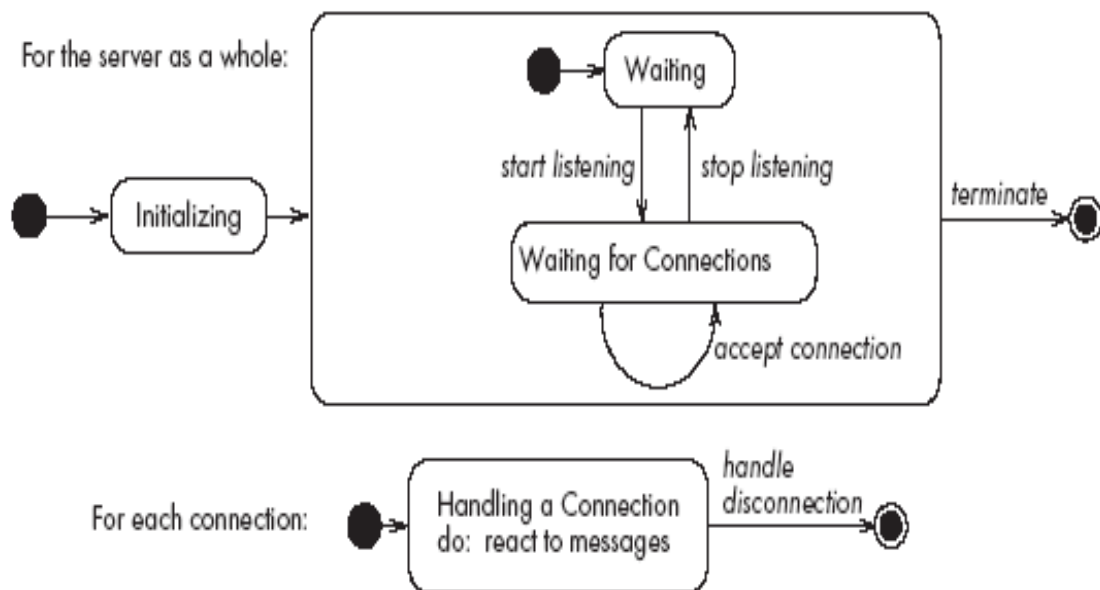


### פעילויות של ה-server

1. אתחול עצמי.
2. להתחיל להקשיב ל-clients.
3. לטפל בסוגים שונים של אירועים הנוצרים על ידי ה-clients.
  - לטפל ב-connections.
  - להגיב להודעות.
  - לטפל ב-client disconnection.

4. להפסיק להקשיב לעיתים.

5. לסיים בצורה מסודרת את הביצוע שלו.

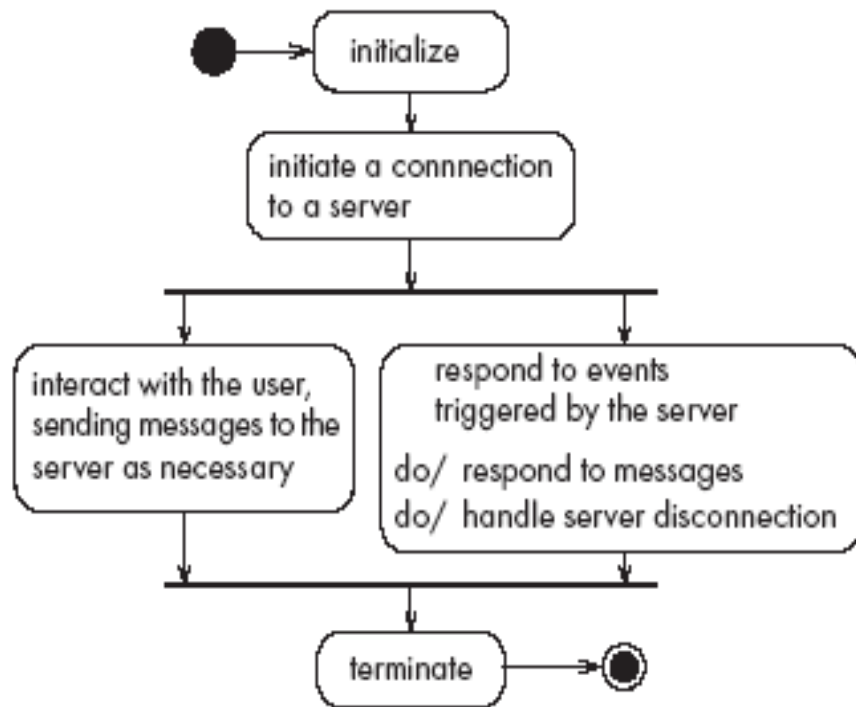


להלן תרשים פעילות של server:

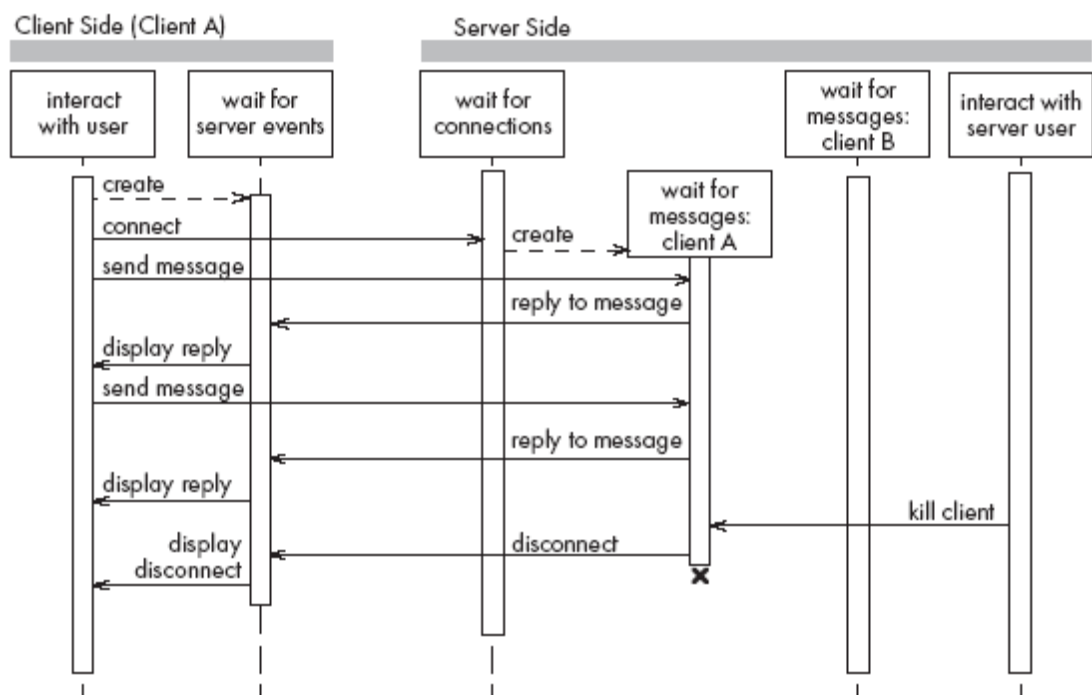
### פעילויות של ה-client

1. אתחול עצמי.
2. להתחיל connection.
3. לשלוח הודעות.
4. לטפל בסוגים שונים של אירועים הנוצרים על ידי ה-server:
  - להגיב להודעות.
  - לטפל ב-server disconnection.
5. לסיים בצורה מסודרת את הביצוע שלו.

להלן תרשים פעילות של client:



חומים במערכת server-client



## Thin-client

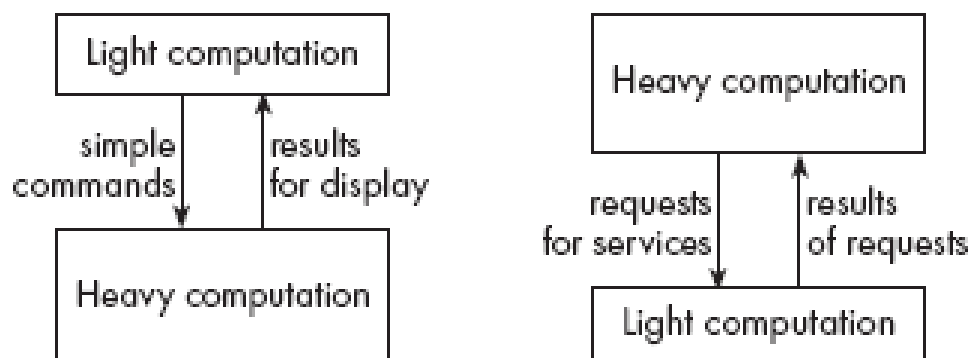
### לעומת fat-client

#### (Thin-client (a מערכות

- ה-client קטן ככל האפשר.
- רוב העיבוד נעשה ע"י ה-server.
- ניתן ל"הוריד" את תוכנת ה-client בקלות באמצעות הרשת.

#### (Fat-client (b מערכות

- ה-client מבצע חלק גדול מעיבוד המערכת.
- ה-server יכול לטפל ביותר clients.



### 3. פרוטוקולי תקשורת

ההודעות המועברות בין ה-clients ל-server יוצרות שפה. גם ההודעות מה-server ל-clients יוצרות שפה. כאשר הם "מדברים" הם משתמשים בשפות אלה. שתי השפות מגדירות פרוטוקול תקשורת.

### 4. משימות שיש לבצע בפיתוח מערכת Client-Server:

1. יש לתכנן את המשימות שעל ה-server וה-client לבצע בתחילת העבודה.
2. יש לתכנן את ביזור עבודת המערכת בין ה-client וה-server.
3. יש לתכנן את ההודעות.
4. יש לתכנן את המנגנונים ל:

1. איתחול
2. טיפול ב-connections
3. שליחה וקבלת הודעות
4. סיום

## 5. טכנולוגיות נדרשות למערכות Client – Server

### (Internet Protocol (IP

- מאפשר משלוח הודעות ממחשב אחד לשני.
- הודעות ארוכות מפוצלות לקטעים קטנים.

### (Transmission Control Protocol (TCP

- מטפל ב- connections בין שני מחשבים.
- מחשבים יכולים להחליף הרבה הודעות IP באמצעות ה-connection.
- מבטיח שהודעות התקבלו בצורה משביעת רצון.

### ל-host יש כתובת IP ושם.

- מספר servers יכולים להתבצע על אותו host.
- כל server מזוהה ע"י מספר (port 0 – 65535).
- כדי להתחיל תקשורת עם server, ה-client צריך לדעת אם מספר ה-port ואת שם ה-host.

## 6. כינון connection ב-Java

החבילה java.net package מאפשרת יצירת TCP/IP connection בין שתי אפליקציות.

The java.net package

לפני הקמת ה-connection, ה-server חייב להתחיל להקשיב לאחד מה-ports:

```
ServerSocket serverSocket = new ServerSocket(port);
```

```
Socket clientSocket = serverSocket.accept();
```

ליצירת connection בין ה-client ל-server

```
Socket clientSocket= new Socket(host, port);
```

## 7. החלפת מידע ב-Java

כל תוכנית משתמשת במופעים של המחלקות (שנמצאות בחבילה java.io):

InputStream – לקבלת הודעות מתוכנית אחרת.

OutputStream – לשלוח הודעות לתוכנית אחרת.

```
output = clientSocket.getOutputStream();
```

```
input = clientSocket.getInputStream();
```

### משלוח וקבלת הודעות

without any filters (raw bytes)

```
output.write(msg);
```

```
msg = input.read();
```

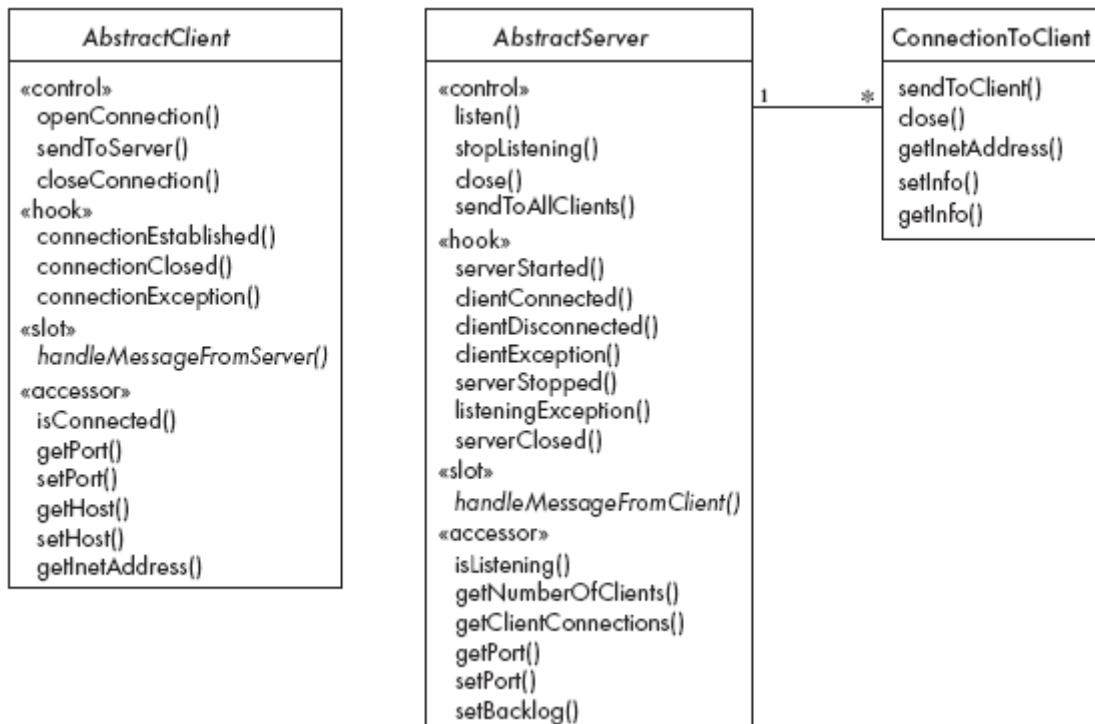
- or using DataInputStream / DataOutputStream filters

```
output.writeDouble(msg);
msg = input.readDouble();
```

- or using `ObjectInputStream / ObjectOutputStream` filters

```
output.writeObject(msg);
msg = input.readObject();
```

### 8. The Object Client-Server Framework (OCSF)



### שימוש ב-OCSF

- אין משנים את שלושת המחלקות של ה-OCSF.
- יוצרים תת מחלקות המרחיבות את המחלקות `AbstractClient` ו-`AbstractServer`.
- קוראים למתודות הציבוריות של ה-framework.
- דורסים מתודות `slot` ו-`hook` (שתוכננו מיוחד לפעולת הדריסה).

### צד ה-Client

מורכב ממחלקה בודדת בשם `AbstractClient`. חובה להרחיבה. כל תת מחלקה צריכה לספק מימוש למתודה `handleMessageFromServer` שמטפלת בקבלת הודעה מה-server.

המחלקה מממשת את הממשק `Runnable` – כלומר יש בה מימוש למתודה `run`

### הממשק הציבורי של המחלקה AbstractClient

Controlling methods:

- `openConnection`
- `closeConnection`

- sendToServer

Accessing methods:

- isConnected
- getHost
- setHost
- getPort
- setPort
- getInetAddress

**מתודות שניתן לדרוס:**

- connectionEstablished
- connectionClosed

**מתודות שחובה לממש:**

- handleMessageFromServer

### **הנחיות לשימוש במחלקה AbstractClient**

יש ליצור תת מחלקה של AbstractClient

יש לממש את המתודה handleMessageFromServer

יש לכתוב קוד ש:

- יוצר מופע של המחלקה המרחיבה את AbstractClient.
- קורא למתודה openConnection.
- שולח הודעות ל-server באמצעות המתודה sendToServer.
- יש לממש את המתודות connectionClosed ו- connectionException.

### **משתני מופע:**

- Socket השומר את כל המידע על הקשר עם ה-Server.
- שני Streams: ObjectOutputStream ו- ObjectInputStream.
- חוט אשר רץ ומבצע את מתודת ה-run של ה-AbstractClient.
- שני משתנים אשר מאחסנים את ה-host וה-port של ה-server.

### **צד ה-Server**

כולל שתי מחלקות:

- **AbstractServer** – החוט אשר מקשיב ל-connections חדשים.
- **ConnectionToClient** - חוטים אשר מטפלים ב-connections ל-clients.



### הממשק הציבורי של המחלקה AbstractServer

Controlling methods:

- listen
- stopListening
- close
- sendToAllClients

Accessing methods:

- isListening
- getClientConnections
- getPort
- setPort
- setBacklog

מתודות שניתן לדרוס:

- serverStarted
- clientConnected
- clientDisconnected
- clientException
- serverStopped
- listeningException
- serverClosed

מתודות שחובה לממש:

- handleMessageFromClient

### הממשק הציבורי של המחלקה ConnectionToClient

Controlling methods:

- sendToClient
- close

Accessing methods:

- getInetAddress
- setInfo
- getInfo
- 

### הנחיות לשימוש במחלקות AbstractServer ו- ConnectionToClient

יש ליצור תת מחלקה של AbstractServer.

יש לממש את המתודה handleMessageFromClient.

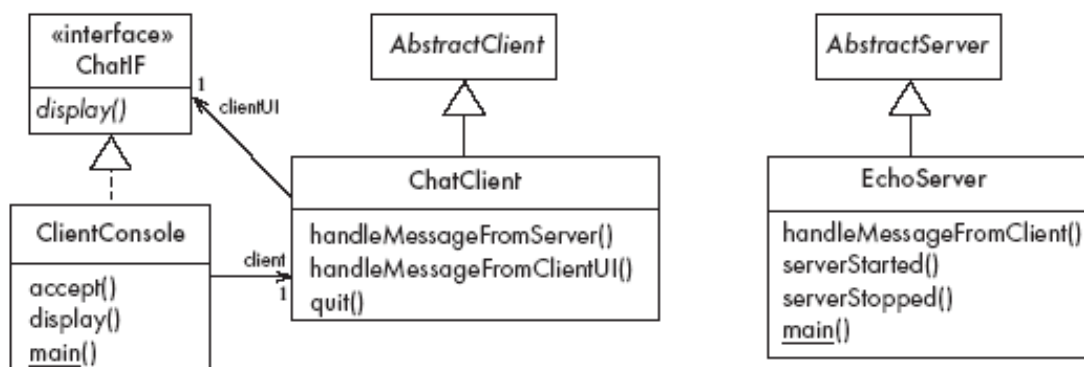
יש לכתוב קוד ש-

- יוצר מופע של המחלקה המרחיבה את ה- `AbstractServer`.
- קורא למתודה `listen`.
- שולח הודעות ל- `clients` בעזרת המתודות `getClientConnections` ו- `sendToClient` או `sendToAllClients`.
- ניתן לממש מתודות נוספות.

### המימוש הפנימי של `AbstractServer` ו- `ConnectionToClient`

- המתודות `setInfo` ו- `getInfo` עושות שימוש במחלקה `HashMap`.
- מתודות רבות חלק של ה- `server` הן `synchronized`.
- אוסף המופעים של `ConnectionToClient` מאוחסן במחלקה מיוחדת הנקראת `ThreadGroup`.
- ה- `server` צריך לעצור את ההקשבה כל `500ms` כדי לראות אם המתודה `stopListening` נקראה. אם לא, יש לחדש את ההקשבה מיד.

### 9. - `SimpleChat` אפליקציה להחלפת הודעות בין client ל- server



### Server-ה

- ה- `EchoServer` הוא תת-מחלקה של `AbstractServer`.
- המתודה `main` יוצרת מופע חדש של המחלקה המרחיבה את ה- `AbstractServer`.
- המופע מקשיב ל- `clients` ומטפל ב- `connections` עד שה- `server` מופסק.
- המתודות `serverStarted`, `serverStopped`, `handleMessageFromClient` מדפיסות הודעה למשתמש.
- המתודה `handleMessageFromClient` קוראת ל- `sendToAllClients` (וזו עושה `echo` לכל הודעה).

### Key code in EchoServer

```
public void handleMessageFromClient
(Object msg, ConnectionToClient client)
{
    System.out.println( "Message received: " + msg + " from " + client);
    this.sendToAllClients(msg);
}
```

### Client-ה

כאשר התוכנית של ה-client מתחילה היא יוצרת שני מופעים של שתי מחלקות:

#### **ChatClient**

זוהי תת מחלקה של AbstractClient הדורסת את המתודה handleMessageFromServer (שקוראת למתודת תצוגה).

#### **ClientConsole**

מחלקה עבור ממשק משתמש המממשת את הממשק ChatIF (מממשת את המתודה display לתצוגה).

במתודה run קוראים למתודה accept לקבלת קלט מהמשתמש ושולחים אותו ל- ChatClient ע"י המתודה handleMessageFromClientUI. מתודה זו קוראת ל- sendToServer.

### **Key code in ChatClient**

```
public void handleMessageFromClientUI(
String message)
{
    try
    {
        sendToServer(message);
    }
    catch(IOException e)
    {
        clientUI.display (
            "Could not send message. " +
            "Terminating client.");
    }
}
```

```
quit();
}
}
public void handleMessageFromServer(Object msg)
{
    clientUI.display(msg.toString());
}
```

### משימות לביצוע

1. הריצו את התוכנית SimpleChat:

- a. הריצו את החלק של ה-Server והחלק של ה-Client על אותו מחשב.
- b. הריצו את החלק של ה-Server והחלק של ה-Client על שני מחשבים שונים.

2. ממשו פקודה 'send' של ה-Client השולחת את פרטיכם האישיים (שם משפחה, ת"ז וכתובת) ל-Server (פקודה מתבצעת אם מכניסים את הפקודה 'send' מה-keyboard).

### הוראות הגשה לדו"ח מעבדה 6:

1. ההגשה בזוגות בלבד באמצעות הגשה אלקטרונית. ניתן להגיש מספר פעמים עד לשעת הסיום.
2. יש להגיש קובץ pdf ובו תצלומי מסך של הפלט של הרצת התוכניות (תצלום עבור הרצה של client server מאותו מחשב, תצלום של שני מחשבים, ותצלום של מימוש של 2). שם הקובץ הוא מספרי ת"ז של הסטודנטים המגישים, מופרדים בגרש תחתון.
3. נא להריץ משני מחשבים שונים, ולצרף צילום דרך הטלפון של זה (יהיה לכם צורך לשנות כתובת ip שה-client פונה אליהם).
4. נא לייצר jar משתי האפליקציות (client, server), ולצרף את הקבצים האלו על מנת להריץ כאפליקציה (בשלב זה אפשר שהIP יהיה local host).

**עבודה נעימה!**